PROJECT-2

Y. HARINI - AI23BTECH11034

R. RISHITHA - AI23BTECH11020

S. RAMA HARSHINI - AI23BTECH11028

Data Sources for project implementation

- For the project, we used 3 real world datasets from Kaggle . Links of those datasets are:

  Steps tracker dataset: https://www.kaggle.com/datasets/monicahjones/steps-tracker-dataset

  Weight-height dataset: https://www.kaggle.com/datasets/mustafaali96/weight-height

  Loan sanction dataset: https://www.kaggle.com/datasets/altruistdelhite04/loan-prediction-problem-dataset

- We used 'Steps_tracker' dataset for both part1 and part2 ,

  Weight-height dataset' for part3 ,

  'Loan sanction dataset' for part4

- The google Collaboratory link in which we have written our python codes
  https://colab.research.google.com/drive/1qV4KweiOk0qaKJBfZqUfMl2rQR3gVCzE#scrollTo=Q3TtQCa_0ESr

- We directly used some results from the class notes without deriving from scratch

Part-1

- Interpreting as Gamma distribution
    1. Method of Moments estimator
    2. Maximum likelihood estimator

- We used the data of the column "water_intake_liters" , interpreting that the data follows Gamma(a,b) distribution we found MOM estimator and MLE of the shape parameter "a" and the scale parameter "b".

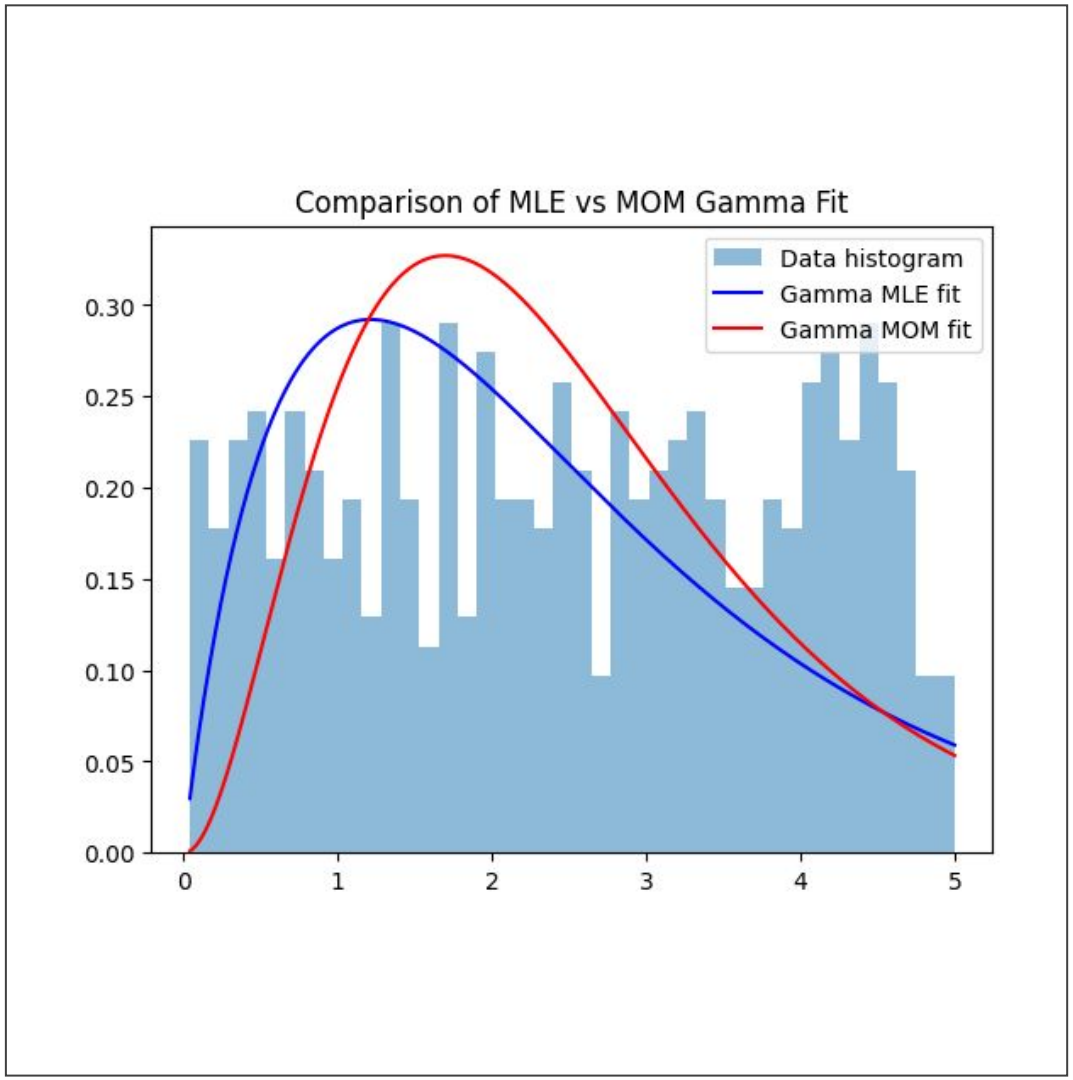
- We can follow similar kind of approach to estimate the parameters for other numerical and positive columns also.

Code for evaluating MOM and MLE estimates for the total amount of water intake (in liters):

```
import pandas as pd
import numpy as np
from scipy.stats import gamma
from scipy.stats import chi2
df = pd.read_csv('steps_tracker_dataset.csv')

# For 'water_intake_liters' column:
#To calculate E(X)
water = df['water_intake_liters']
mean_water = np.mean(water)
print("E(X) = ", mean_water)
#To calculate E(X^2)
var_water = np.var(water)
print("var(X) = ", var_water)
mean_sq_water = var_water + mean_water**2
print("E(X^2) = ", mean_sq_water)
#calculatng MOM etimators of parameters a and b
b_mom = mean_water/var_water
a_mom = mean_water*b_mom
print("MOM estimator of a = ", a_mom)
print("MOM estimator of b = ", b_mom)
a_mle, loc, b_mle = gamma.fit(water, floc=0)
print("MLE estimator of a = ", a_mle)
print("MLE estimator of b = ", b_mle)
```

Output:

```
E(X) = 2.50658
Var(X) = 2.0302509036000003
E(X^2) = 8.3131942
MOM estimator of a = 3.09466346512109
MOM estimator of b = 1.234615877060014
MLE estimator of a = 1.9348860590472108
MLE estimator of b = 1.2954664634021429
```



Conclusions:

◦MOM ,MLE estimated values for shape parameter (a) are 3.0947, 1.9349 .

◦MOM, MLE estimated values for scale parameter (b) are 1.2347 , 1.29547 .

◦The red curve peaks higher than that of blue which tells that MOM overestimates the shape parameter. This may be due to as the MOM may not be able to capture the skewness and discrepancies in the data.

◦We can also see that the data we used is not at all following Gamma distribution and it has so much discrepancies, so we can expect difference in the estimated values using MOM and MLE methods.

Part-2

- Interpreting the data as normal distribution with unknown mean and unknown variance
- Finding the 95% confidence interval for the variance parameter of the data
- As mentioned in the question , we used the previously used "water_intake_liters" column data for this part also
- We directly used the results from the class notes without deiving them explicitly from scratch

Code for evaluating confidence interval for the total amount of water intake (in liters):

```
#For 'water_intake_liters' column:
svar_water = np.var(water, ddof = 1) #degrees of freedom = 1 calculates sample variance
confidence_interval = 0.95
alpha = 1 - confidence_interval
n = len(water)
chi_sq_lower = chi2.ppf(alpha/2, n-1)
chi_sq_upper = chi2.ppf(1-alpha/2, n-1)
lower_bound = (n-1)*svar_water/chi_sq_upper
upper_bound = (n-1)*svar_water/chi_sq_lower
print("Confidence interval for variance of water intake: ", "(", lower_bound, ", ", upper_bound, ")")
```

Output:

```
Confidence interval for variance of water intake: ( 1.8037391582936497, 2.3123690767391856 )
```

Conclusions:

- The 95% confidence interval for the variance parameter of the data is (1.804 , 2.312)
- From the above result we can say that the unknown variance will be present in the calculated interval with 95 percent confidence
- As we know (n-1)s^2/sigma^2 follows chi square(n-1) , using this we evaluated the confidence interval for the variance where s^2 is the sample variance

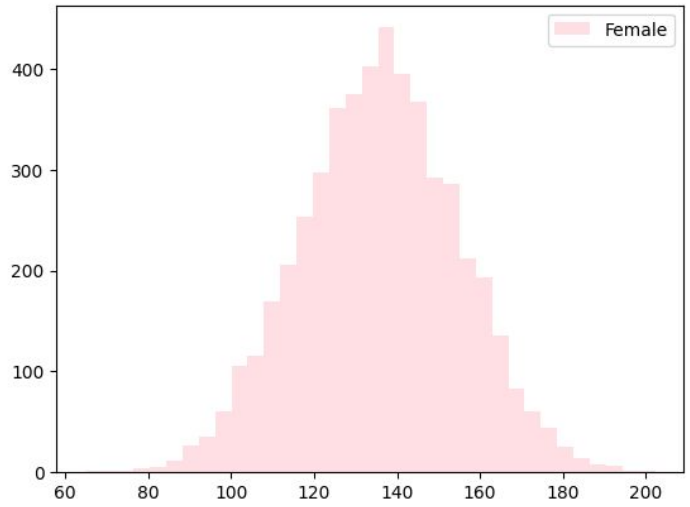
Part –3 (Finding confidence interval)

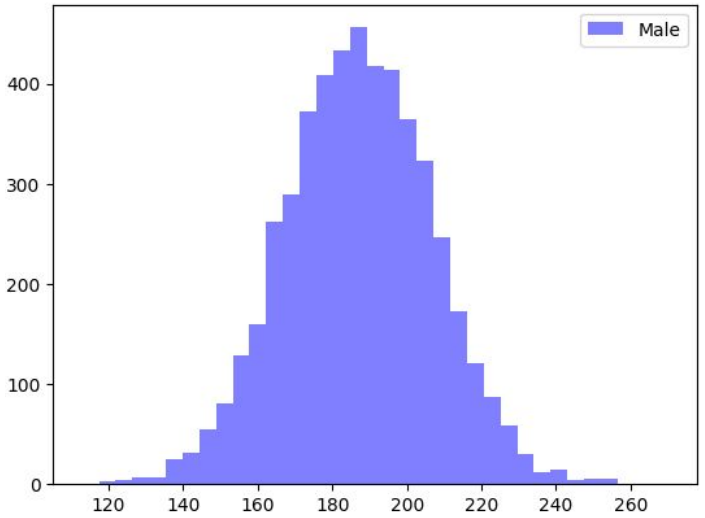
- Interpreting two independent distributions as normal distributions with unknown means and variances and finding 95% confidence interval for the difference in the means of the two populations
- From "weight-height" dataset , we used the weights column and distributed it into two sets one containing the weights of males and another containing the weights of females. We interpreted these weights as two independent normal distributions with unknown means and variances and using statistical approaches we found 95% confidence interval for the difference in the means of two populations(As the weights of males and weights of females have same units, we can subtract mean of one random variable from the mean of the other one)

```python
from scipy.stats import t
import pandas as pd

data = pd.read_csv(path)   #path is the variable which contains the path of the csv file
Male_weights = data[data['Gender'] == 'Male']['Weight']  #Storing the weights of all males in Male_weights
Female_weights = data[data['Gender'] == 'Female']['Weight']  #Storing the weights of all females in Female_weights

sample_mean_male = Male_weights.mean()
sample_mean_female = Female_weights.mean()
sample_variance_male = Male_weights.var(ddof=1)
sample_variance_female = Female_weights.var(ddof=1)
sample_mean_diff = sample_mean_male - sample_mean_female

m= len(Male_weights)
n= len(Female_weights)
S_c = ((m-1)*sample_variance_male + (n-1)*sample_variance_female)/(m+n-2)

# Upper alpha point of t distribution with degree of freedom d
def upper_alpha_point(alpha, d):
    return t.ppf(1 - alpha, df=d)

alpha = 0.05
t_ = upper_alpha_point(alpha/2,m+n-2)
lower_bound = sample_mean_diff - t_ * (S_c * (1/m + 1/n))**0.5
upper_bound = sample_mean_diff + t_ * (S_c * (1/m + 1/n))**0.5

print("Lower bound of interval:",lower_bound)
print("Upper bound of the interval:",upper_bound)
```

Output:

```
Lower bound of  the interval:  50.39975317405964
Upper bound of the interval: 51.92130212739005
```





Conclusions:

- As we have 5000 samples of male weights and 5000 samples of female weights, we can observe the hists of the weights of male , female are individually approximately normal. So our interpretation helps well for calculation of the confidence interval.

- Calculating the 95 percent confidence interval for the difference in mean of male and female weights, we get the lower bound of the interval as "50.3998" and upper bound of the interval as "51.9213"

- So the 95 percent confidence interval is (50.3998, 51.9213)

Part 4

- Hypothesis testing for real world bernoulli data

- We used the data of a bank which approved loans to some people and didn't approve for remaining people. We used the particular column 'Loan Status' which contains 'Y' if loan is sanctioned, 'N' Otherwise. Using this data we created another list which contains 1 to the corresponding 'Y' and 0 to the corresponding 'N'. Interpreting it as bernoulli data , we performed hypothesis testing with the given H_0 and H_1 at a significance level of 0.05 as mentioned.

Code for Hypothesis testing of bernoulli random variable:

```
from scipy.stats import binomtest
import pandas as pd

data = pd.read_csv(path)  # path contains the path of the csv file that contains data
loan_status=data['Loan_Status']
loan_approval_data = (loan_status == 'Y').astype(int)

n = len(loan_approval_data)
print("Total length of the data: ",n)      # sample size

x = np.sum(loan_approval_data)
print("No.of people whose loan is approved:",x)      # number of successes
p0 = 0.5          # hypothesized success rate
print("Probability of a person recieving loan =",x/n)

#Calculating TDF of binomial(n,p0) at x
result = binomtest(x, n, p=p0, alternative='greater')
print(f"p-value: {result.pvalue}")
```

Output:

```
Total length of the data:  614
No.of people whose loan is approved: 422
Probability of a person receiving loan = 0.6872967
p-value: 4.248833642487421e-21
```

Conclusions:

- Given H_0: $P_0 <= ½$, H_1: $P_0 > ½$ and level of significance is 0.05.
- The p-value we obtained is `4.248833642487421e-21`
- Since the p-value is very less than significance level (0.05), we reject the null hypothesis. This provides strong statistical evidence that the true proportion of loan approvals is greater than 0.5 (P_0).
- The data strongly suggests that more than 50% of applicants are approved for loans at a significance level of 0.05. So we reject H_0 and accept H_1 at a significance level of 0.05

THANK YOU

- Presented by

- Rishitha
- Harini
- Rama Harshini